

New implementation of sets in New Stanford Pascal

First of all: please excuse possible errors in my English; I am German and not a native English speaker ... I will do the best I can.

A Moshix video of 2023 tells about the successful installation of New Stanford Pascal on Hercules, look here: <https://www.youtube.com/watch?v=li2J4K2JwtE>

But when Moshix tried to write and compile his 8-queens program, it failed.

As it turned out, the reason was that the program used a set with a base type containing negative integers like the following:

```
program QUEENS ( OUTPUT ) ;

const CMIN = 1 ;
      CMAX = 8 ;
      DMIN = - 7 ;
      DMAX = 7 ;

type CRANGE = CMIN .. CMAX ;
   COLUMN = set of CRANGE ;
   FIELD = array [ CRANGE ] of CRANGE ;
   DRANGE = DMIN .. DMAX ;
   DIAG = set of DRANGE ;
```

see the type DIAG, which is in fact a set of -7 .. 7

(I think the type DIAG is used to record which diagonale of the chess board is covered by the queens already on the board ...)

The Compiler of 2023, as it turned out, didn't support sets with negative integers. It didn't complain, but silently generated wrong code.

That's why I had to invest some time and energy in finding a new solution for sets.

Let me first describe, how sets were implemented prior to 2024.

Sets before 2024

I already did some improvements to sets, but I did not address the basic shortcomings. In Stanford Pascal, the sets before 2024 are all stored starting with the element zero, no matter if they are supposed to have negative elements (they are simply ignored) or if they start at a higher element, as in set of 10 .. 20.

A set is implemented as a bit list, that is, there is a bit for every element in the set.

The leftmost bit of the set is the element zero (if the base type is integer) or the first value of a scalar type (which is internally represented as zero). The next bit is one, the following two and so on.

In 2023, the sets were limited to 2000 bits, which makes ca. 250 bytes.

The sets in storage had no length informations; the length informations are part of the set instructions. We have SLD (set load), which loads a set from storage to the stack (that is: builds a set descriptor, see below), and then SMV (set move), which assigns sets, even of different lengths. And we have UNI for union of sets, INT for intersection and so on ...

When sets are processed, a sort of „set descriptor“ is transferred to the stack, which consists of a 3 byte address of the set (in storage) and a 1 byte set length (in bytes). This 1 byte length limits the sets to 2000 elements, and the 3 byte address is a problem, BTW, for the upcoming AMODE 31 support of the mainframe variants of New Stanford Pascal.

So there is enough motivation to design a „new“ implementation of New Stanford Pascal sets.

New Sets of 2024

The sets must support negative starting points (and positive starting points, which are not zero). And: the length information should not be part of the set descriptors any more, so that there is no 3 byte limit for the addresses.

Otherwise, if the length information stayed in the descriptors, they would have to be expanded to 8 bytes, but this would have a much bigger overall impact on the compiler etc.

The soundest solution IMO is:

The set representation in storage gets 4 more bytes (at the beginning); the first two bytes is the net length of the set representation (number of bytes after the 4 byte prefix). The next two bytes contain the set origin, that is, the internal value of the first bit of the set representation. This should always be a multiple of 8 (and: can be negative, of course).

Two examples:

a) set of 10 .. 30

this will start at element 8 internally and will need 3 bytes – in fact 7 because of the 4 byte prefix (elements 8 thru 31)

0003 0008 xx xx xx

b) set of -7 .. 7 (from the chess example)

starts at -8 and needs only two bytes (in fact 6)

0002 FFF8 xx xx

An optimization comes to mind:

because all set origins are multiples of 8, it is possible to record the set origin divided by 8 in the second field of the prefix. This way base types in the range -262.136 to +262.136 are possible. Using only positive lengths for the net length, the difference max – min may not exceed 262.136. We'll start with that; that's a hundredfold increase to what we had in 2023.

So the two examples above will be changed like follows:

0003 0001 xx xx xx

0002 FFFF xx xx

Smooth migration

The problem now is:

how can I change the set implementation, when the sets are heavily used inside the compiler itself? If I change the implementation of the sets, the compiler will not be able to compile itself.

The only solution to this problem IMO is: the new implementation of sets must rely completely on new P-Code instructions, which are different from the existing instructions. The old P-Code instructions must work the same way as before, so that the compiler can continue to use them, and the P-Code interpreter (or 370-translator) can be extended to support the new P-Codes, without discontinuing support for the old P-Codes.

The existing compiler works with the existing instructions and is used in the first stage to compile test programs only. Then these test programs (like the Moshix 8-queens program, for example) are tested, until they work correctly.

And then (stage 2), the new compiler is used to compile itself.

If this works – with the new set implementation – the old P-Code instructions can be phased out.

To do this, I have to identify in the first place, which P-Code instructions need changes.

The first change is related to the definitions of sets, that is: the calculation of the lengths of set types and variables and the definition of set constants in P-Code.

This is an example of two set constants in P-Code today:

```
16 DFC S,X16'008181818181818181818181818181'
32 DFC S,X28'008181818181818181818181818181818181818181818181',
    '8181818181'
```

The set constants (and variables) have been defined like this:

```
type SETX = set of - 100 .. 100 ;
      SETY = set of 100 .. 200 ;

var X : SETX := [ ] ;
    Y : SETY := [ ] ;
```

This is an error, because the constant should contain all hex zeroes. Furthermore, the first constant contains only 128 bits ($16 * 8$) and the second one 224 ($28 * 8$) – you see the problem with the zero origin. The compiler ignores the -100 origin from the first set type and the +100 origin from the second one completely and starts both sets at origin zero.

The compiler (of 2023) reserves 16 bytes for X and 28 bytes for Y.

The set constants, as shown, are the initialization patterns; the initializations must be carried out at run time (the variables are automatic variables).

The instructions which do this look like this:

```
LOC 39
VPU 1,452
LCA P,$PASMAI#
INC A,16
LDA 1,408
MOV -44
```

This is interesting, because a simple block move is used to do the initialization for both variables, which means that this mechanism should also work later for the new set implementation.

Now, how should the definition of the variables etc. look today with the new compiler?

Given these definitions:

```
type SETX = set of - 100 .. 100 ;  
      SETY = set of 100 .. 200 ;  
  
var X : SETX := [ ] ;  
     Y : SETY := [ ] ;
```

the new set implementation should do the following:

- reserve a bit string from -104 (next lower multiple of 8) to +100 for SETX, which means 205 bits, which means 26 bytes – plus 4 for the prefix, which makes 30 bytes.
- reserve a bit string from 96 to 200, which means 18 bytes (including the 4 bytes prefix)
- built an initialization pattern for X and Y in the constant area, which allows a simple block move during the procedure startup phase, that is:

```
16 DFC S26,-104,X'0'  
46 DFC S14,96,X'0'
```

This is a new syntax for set constants – the number after the S is the length of the sets (stored in the first two bytes of the prefix), then the internal value of the leftmost bit (which is stored divided by 8) and then the bit string in hex notation, which is padded to the right by zeroes, up to the length from above.

Using this notation, it would be possible to support both notations (the old one and the new one) at the same time by P-Code interpreters and translators (to support a smooth migration).

Please note that the internal representation of such „new“ set constants depends on the platform (because of the numbers in the prefix, which are subject to endianness issues). What I show here, therefore, is the mainframe representation:

```
000010: 001A FFF3 000000 ...  
00002E: 000E 000C 000000 ...
```

Note:

X'FFFE' = -13 (dec.) which means -104

X'000C' = 12 (dec.) which means +96

because to get the real set origin, you have to multiply the number stored by 8.

I have to add one another important remark:

To support the initialization above (by block move), the patterns in the constant area (which are empty sets, after all) have to be created twice in different format, because of the different sizes and origins of the two set types.

But ...

in normal operation, when assigning empty sets to set variables, we can use another simple representation of the empty set, which is invariant of the target set type.

This representation is:

0000 0000

That is:

length of the bit string is zero, and the set origin is zero, too.

When assigning this set to a set variable, we have to create a NEW set instruction (which we have to do anyway), which respects the different lengths of source and target AND the possible different set origins.

Let's assume, that we assign an empty set (which looks like the pattern above) to a set of type SETY from above, that is:

```
type SETY = set of 100 .. 200 ;
```

```
var Y : SETY ;
```

```
...
```

```
Y := [ ] ;
```

In this case, the origin of the source set (which is zero) is OUTSIDE the set margins of the set type SETY ... but IMO, this doesn't matter as long as there are no set elements in the source set which need to be transferred and which would be outside of the margins of the target set.

Same goes for a source set which has LARGER margins than a target set. An assignment IMO is valid, as long as all elements of the source set fit into the target set. (Maybe a runtime check could be inserted, depending on a compiler option ... we'll see later).

So, after contemplating some time about the layout of sets in storage and about how new set constant definitions should look, I already changed the compiler to create set constant definitions (in P-Code) like designed above, and now it comes to the next step.

Defining new P-Code instructions

The existing P-Codes which deal with set constants are DFC and LCA.

They can both be kept, IMO, because the new set constants can be distinguished from the old ones by the decimal length which immediately follows the letter S (for „set“). This way, it is clear that there is the set origin after the comma, and then the X (hexadecimal) set representation. If there is no length after the S (that is, if the comma follows immediately), we have the old syntax.

So it is possible for the P-Code interpreters or -compilers to handle both the old and the new syntax.

The next task deals with the two instructions SLD and SMV (set load and set move).

Here we have the old description of these instructions (from the 2019 P-Code paper):

SLD = set load

This instruction has two parameters: it loads a set constant or variable to the stack for further processing. p (the first parameter) is the length of the set in bytes, and q is the target address. The binary representation of the set (the source) is found at the address which is on top of the stack; this address was placed there before. The SLD instruction pops this address, moves p bytes from the source address to q, and pushes p and q.

SMV = set move

This instruction has two parameters: it moves a set from the stack to a set variable in storage. Both parameters (p and q) are length parameters. If the second length is lower than the first length, only the number of bytes specified by the first length is moved. The rest of the bytes of the target set variable is set to zero.

The sign of p may be negative or positive.

If p is positive,

- before the execution of SMV, the set (address and length) is on top of the stack; the target address is at the second position.

If p is negative,

- the absolute value of p is used as first length,
- before the execution of SMV, the target address is on top of the stack and the set (address and length) is at the second position.

SMV pops all two stack items in both cases.

An example:

[illegible]

This is old syntax, of course.

In this example, you can see a constant set of length 20. Its address is first brought to the stack by LCA, then a SLD instruction is needed to build the old set representation on the stack (which consists of set address AND set length) and then this set is moved to another places; the address of the target has already been placed on the stack by LDA.

Using the new set representation, we can do much better, IMO:

```
LOC 46
LDA 1,440 ; Y
LCA S4,120,X'80000002'
SMN 20,96
```

SMN means „set move new“.

There is no need for SLD any more in this case, because we have no special representations of sets on the stack; the set is simply identified by its address, and as long as the set is read/only, it can remain in the constant area.

Here the set is copied out of the constant area, and this is done using the new SMN instruction. It uses the addresses on the stack and it has two parameters, both rely to the target. The first parameter is the length of the target set (including the four bytes of meta information at the beginning) and the second byte is the set origin of the target set.

Both operands of SMN can be derived directly from the type of the target set. The implementation of SMN has to deal with different set origins (which means that the set representation of the source set has to be shifted correctly) and it has to check that all elements of the source set fit into the target set. More on this later ...

Now: to support my idea of a smooth migration, I would like to create an intermediate compiler which does the following:

- create new set constants with origins etc. as outlined above
- but still create the old SLD and SMV instructions, with the lengths corresponding to the old set types (which don't have set origins, but instead all set elements stored starting with the element zero)
- then the implementation of the SLD instruction should be changed such, that it translates the LCA set constants of the new format to the old representation

But:

it just comes to my mind that this will not work, because SLD does not know if it has to load a set constant (which has the new format) or a set variable (which has the old format).

Let's see, if I can clone the SLD instruction into two different instructions; one for constants and one for variables ... and then only the constant instruction must be changed to handle the new set constant format.

My goal is to create an intermediate compiler which incorporates some of the changes I made and still works :-). the best test of the compiler is to see if it compiles itself and produces the same code as its predecessor.

Some days later:

I decided not to go this path any further, because I already created the new set format for the DFC constants, and they are moved to set variables (using block move instructions), so it is not possible any more to have a different format for set constants and set variables. BTW: I discovered this, AFTER establishing a new P-Code instruction SLC (= set load constant), which does the same as SLD, but for set constants instead of set variables. This is all moot, because IMO both instructions will not be needed any more in the near future.

Next steps:

- I'll generate the new instruction SMN instead of SMV, using the type attributes of the target
- The SLD and SLC instructions will be kept, but will be eliminated in the near future
- The problem is with the calls to FORCETEMPSET; this copies the set (constant or variable) to a temporary buffer and puts its address on the stack (using SLD/SLC); this is done today every time when a set is checked, but in the future, it is needed only during set manipulations (using + and * operations), but not – for example – for a simple IN check.
- Set moves between variables of the same set type could be done with simple block moves.

- Fix a simple error: the length of the set constants in the DFC instructions includes the 4 byte meta information, but the length in LCA S does not. It is better to NOT include the length of the meta information; so the length information only holds the length of the REAL set bit list.

List of existing set-related P-Code instructions

- ASE = add element to set
- ASR = add range to set
- CHK = check stack item (must be verified)
- DFC = define set constant (already modified ...)
- DIF = difference of two sets
- EQU = equal comparison (set variant ... same for other comparisons)
- IND = indirect access (set variant must be verified)
- INN = check if element is in set
- INT = intersection of two sets
- LOD = load (set variant must be verified)
- RET = return (can it really return sets?)
- SCL = set clear
- SLC = set load constant (new, see above)
- SLD = set load
- SMV = set move
- UNI = union of two sets

As mentioned above, the existing set instructions shall remain unchanged during the migration process, so that the existing compiler still works, using the old P-Code instructions. I already mentioned a new P-Code instruction called SMN (set move new) above, but now I am thinking of renaming it and putting all the new instructions in a separate „name space“ by giving them a new starting letter. Z comes to mind; we don't have instructions starting with the letter Z at the moment. So I would like to consolidate all new set instructions as instructions with the letter Z.

But there are some exceptions. First of all, I have to check, if some of the instructions above really work with sets (for example IND, LOD and RET etc.). Second, LCA, DFC and the compare instructions are already known to work with sets as well as with other types and should work with old and new set representations simultaneously.

New set-related P-Code instructions

At the moment, we plan the following new set instructions:

- ZAE = add element to set
- ZAR = add range to set
- ZDI = difference of two sets
- ZIN = check if element is in set
- ZIS = intersection of two sets
- ZCL = set clear
- ZMV = set move
- ZUN = union of two sets

I don't know yet, if all of these instructions are really needed. And if some instructions are still missing.

The next steps:

- rename SMN to ZMV
- create ZIN instead of INN
- omit the SLC and SLD instructions
- write a new PCINT version, which implements the instructions so far, and look if it handles the test programs TESTSET9.PAS and TESTSETA.PAS correctly (this will sure take some days or weeks)

Builtin functions

When working on the new P-Code instructions. I discovered some other areas which need rework:

- the CARD function has a set argument
- see procedure GENSETOP; it generates the set operations (new P-Codes) ZIS, ZDI and ZUN ... but it also does other work on the stack representations, which maybe is not needed any more or needs changes ...

I thought about telling somewhere in the P-Code if the compiler uses the old or the new set implementation ... this way the interpreters and P-Code translators would know how instructions that stay the same (like the comparison instructions etc.) should be handled.

A flag which tells this could be included in the function prolog, that is: in the P-Code instruction ENT, if there is space remaining. Otherwise, another new mode instruction, which shows several flags for the compile unit, could be established.

This flag could also control the behaviour of the builtin functions mentioned above.

02.09.2024

In the meantime, I changed the compiler to output the ZMV and ZIN instructions (and DFC and LCA for sets, of course), and I changed PCINT to understand them. Now the test programs TESTSET9 and TESTSETA work correctly, even for sets involving negative integers. But some problems still remain, because sets with base type CHAR are not handled correctly. I am trying to solve this in the next days.

Every set involving characters should only create constants using the C notation (because of portability concerns); this is also true if the base type is not CHAR, but only a subrange of CHAR. This was not correct, as I observed this morning, but it could easily be fixed by introducing a new function IS_CHAR_TYPE (of type BOOLEAN), which returns true not only for CHAR types, but also for subranges of CHAR. This function must be used whenever sets have to be examined for a base type involving CHARs.

07.09.2024

I needed an additional new P-Code instruction. ZMX is much the same as ZMV, but it leaves the address of the result on the stack for further processing. This is needed prior to the new instructions ZUN (set union), ZIS (set intersection) and ZDI (set difference), because in these situations one of the sets involved in the operation has to be copied to a work area first, and this is done using ZMX.

Today I finally have a working version of the compiler which produces correct code for ZUN, ZIS and ZDI operations ... and they also work and give correct results with PCINT. This is a major breakthrough and shows that my design is valid – so far.

When trying to compile the compiler, I get some problems with such statements:

```
ERRLOG := ERRLOG + [ FERRNR ] ;
```

where ERRLOG is a set with base type INTEGER and FERRNR is an integer variable. At the moment, the new compiler abends, when compiling this ... the old compiler compiled this without problems.

This must be fixed in the next step ... the compiler tries to check the properties of the right side operand of the set union, but obviously it can't do it, because the value of FERRNR is not known, and so there are no properties (set origin and length etc.). The old compiler had no need for set properties ... all sets started with element zero, so it could implement this without requiring information about the set properties (maybe the element was outside the set ... but who cares).

The solution to this is the (former) P-Code instruction ASE, but this instruction must get a new equivalent, as the other old set instructions, which are all obsolete now.

Same goes for expressions with a mixture of constants and variables; let's assume, the expression above would look like this:

```
ERRLOG := ERRLOG + [ 12, FERRNR, 15 ] ;
```

or like this:

```
ERRLOG := ERRLOG + [ FERRNR1 .. FERRNR2 ] ;
```

More to do in the next weeks :-)

The rework of the set implementation is finished, when the compiler is able to compile itself and it is successfully running (that is, it can repeatedly compile itself and produce identical results – and the compiler uses Pascal sets heavily; for example: the symbols which may restart parsing after a syntax error in a certain situation are stored in a Pascal set).

18.01.2026

I finally continued the work on the compiler after a long break. I was very busy with my money jobs in the meantime. What motivated me: Marc Waterbury sent me a LISP interpreter, written for Pascal/VS, which I ported in short time to Stanford Pascal on the PC. But to get it running successfully on VM and MVS, I have to add new diagnostic features to the compiler, and this does not work, if the compiler is in an unstable (developing) version. So I decided to first complete the work on sets and THEN to add new features to the compiler.

The first thing I did was to add a section at the end of this document which describes the new P instructions, as they are implemented so far.

The next step will be to implement the instructions ZAE and ZAR in the PCINT P-Code emulator, to see, if the test program TESTSETB will run correctly; this test program, by the way, runs almost correctly with the old compiler, it only omits negative values from the (integer based) sets, which it shouldn't do. I hope that TESTSETB will behave as it should, once the new instructions are implemented,

21.08.2026

After another long break I restarted the work on the compiler today. At the end of June I lost one of my customers, which is OK, because I am 67 years old now. I have worked more than 6 years with that customer. Now I will maybe have more time to work on the compiler. And I bought a new laptop yesterday, so the development work may be more fun.

It took me some time to find out where to start, but as it seems, the next step will be to implement the two missing instructions ZAE (set add element) and ZAR (set add range). I recall how they are supposed to work, but the details are somehow unclear. Let's see tomorrow.

22.08.2026

First of all, I started with checking the old test cases. When some of the new set instructions are generated, there are some LDA instructions generated, too, which are not necessary, IMO. I will examine and try to fix that. Another issue could be that there is temporary storage needed for complex set instructions, but I am not sure how to allocate this storage and if it can be freed in any case at the statement boundary ... to examine this further, I added some trace messages in the PCODE output which show me the level of the temp storage used at any given point ... let's see how this turns out ...

23.08.2026

The key problem with the implementation of the new instructions ZAE and ZAR seems to be: they must build a new set without known type information. And: they have to build it in the temporary area following LCOUNTER and leave the address on the stack, much the same as ZMX does it, so that the address of the set can be used for further processing (for example: assigning the resulting set to a target).

The compiler, at the moment, does not build correct LDA instructions before ZAE and ZAR for the target address of the set on the stack, instead it throws out LDA addresses after ZAE and ZAR. This needs to be understood and fixed, before both instructions can be implemented.

25.08.2026

Now I'm again on track, and I believe I understood the ideas I had in 2024. Let's assume we have a statement like the following:

```
Z := [ 12 , INTVAR1 , 20 , INTVAR2 .. INTVAR3 , 50 ] ;
```

Here a set is assigned to the set variable Z; the set is built by a mixture of variables (including a range) and constants.

The idea in 2024 was to build a set constant, consisting of the constant parts of this expression, while doing the parsing. But, for the variable parts. ZAE and ZAR instructions are thrown out „on the fly“; the different temporary sets created by ZAE and ZAR are combined using ZUN. In a final step, the resulting set of the variable parts is then „ZUNed“ with the set constant, giving the final result.

This is the most complicated case; simpler cases are:

- if there are no variables, we only have one set constant (which is already implemented and works correctly)
- if there are no constants, we have only ZAR and ZAE instructions and there is no need to ZUN a set constant in the final step

With these ideas, the assignment above now is compiled as follows:

```
;---- Neues Statement - LCOUNTER-Pegel = 512
LOC 100
LDA 1,440                ; Z
LOD I,1,496              ; INTVAR1
LDA 1,512
ZAE I
LOD I,1,500              ; INTVAR2
LOD I,1,504              ; INTVAR3
LDA 1,544
ZAR I
ZUN
LCA S6,8,X'080800000020'
ZUN
ZMV 52,-104
```

Note, BTW, that it is now possible to write comments to the PCODE output by the compiler; this way providing trace output directly into the PCODE file. PCINT (the PCODE interpreter) has been enhanced so that it ignores lines which start with a semicolon.

The next task now will be to implement the ZAE and ZAR instructions in PCINT and see if the test programs work as expected.

There is another „problem“ which showed up when looking at the PCODE output; look this example:

```
Z := X + [ INTVAR1 .. INTVAR2 ] ;
```

This is compiled as

```
;---- Neues Statement - LCOUNTER-Page1 = 512
LOC 96
LDA 1,440                ; Z
LOD I,1,496              ; INTVAR1
LOD I,1,500              ; INTVAR2
LDA 1,512
ZAR I
LDA 1,408                ; X
LDA 1,33012
ZMX -28,-104
ZUN
ZMV 52,-104
```

Here the set X is copied to the work area (here: address 33012) before being modified by ZUN. This is correct and necessary at the moment, because the set X must not be changed by the following ZUN. But: the second set involved in the ZUN operation is already in the work area (at position 512) and could be used as target of the ZUN operation. But this is not possible (at the moment), because it is the second operand on the stack (not on the first position). The solution would be a ZUN operation which can write the result to the second operand on the stack, not the first.

25.08.2026 (again)

Now I will try to start the implementation of ZAE and ZAR in PCINT; I first added a few remarks in the „instruction section“ at the end of this document, which show the problem. They are also repeated here:

ZAE – replaces ASE – add set element

not yet implemented, needs to be done – see TESTSETB testcase

The problem with ZAE (and ZAR) is: a new set has to be constructed in the work area (after LCOUNTER), and we have no description for this set (no type), because the value of the integer, char or subrange variable which builds this set is not known at compile time.

ZAR – replaces ASR – add set range

not yet implemented, needs to be done – see TESTSETB testcase

Same problem as with ZAE, but much harder; ZAE builds a set consisting of one element, so the set is not very large, given the new offset capabilities; but a set built by ZAR can be VERY large; it consists of all elements between two range limits, and so the size for the temporary area for the ZAR target must be MAXSETL (compiler constant).

Both instructions (see examples on prior pages) have the set target address (which is in the working area) at the topmost stack position and the value of the integer or char value which is used to build the set at the 2nd stack position (if ZAR, left limit on the 3rd stack position and right limit on the 2nd stack position).

The result of the instructions should be:

all integer operands should be popped, only the address of the set remains on the stack (which is moved 1 position by ZAE and 2 positions by ZAR). The set is constructed as needed by ZAE and ZAR.

BTW: the needed space for the set is reserved in the working area by the compiler (it sets the LCOUNTER value for subsequent references to the working area).

LCOUNTER is reset at the beginning of the next statement, because the sets in the working area are not needed across statement boundaries. see how this turns out ...

26.08.2026

The ZAE instruction is implemented, and the first statements involving it ran correctly :-)

But there is already a problem:

a statement involving two ZAE instructions is giving the wrong result, although the statements generated in the PCODE file seem correct. Look here:

```
Z := [ INTVAR1 , INTVAR2 ] ;  
WRITELN ( 'should show 35, 40' ) ;  
WRITE_SET ( Z , FALSE ) ;  
WRITELN ( STRICH ) ;
```

is translated to:

```
;---- Neues Statement - LCOUNTER-Peget = 520  
LOC 97  
LDA 1,440                ; Z  
LOD I,1,496              ; INTVAR1  
LDA 1,520  
ZAE I  
LOD I,1,500              ; INTVAR2  
LDA 1,552  
ZAE I  
ZUN  
ZMV 52,-104
```

A set which is built of two variables is created using two temporary sets and then combined via ZUN ... OK, the working area reserved for such sets is too large, 32 bytes at the moment; can be reset to 8 bytes later, but that's a minor topic.

Let's see what goes wrong here ...

I guess I understand it already ... the ZUN instruction uses the attributes of the first operand set to build the attributes of the result. BUT: it should probably EXPAND the range of the result set, if the second operand set has elements which are outside the range of the first operand set :-)

I will examine this using the trace facilities of PCINT and see how I can correct the way ZUN works. This is because the dynamic sets created by ZAE etc. have no types a priori.

BTW: this is a problem only for ZUN; should be no problem for ZIN (intersect) or ZDI (set difference). In these cases it is OK IMO to derive the attributes of the result from the attributes of the first operand (because elements only „go away“ in these cases).

27.08.2026

After thinking some time about the problems with ZUN, it came to my mind that the sets on the work area should not be allocated with fixed positions and fixed length at compile time. At compile time it is simply not known what the limits of sets are, which are built by ZAE or ZAR instructions or especially a combination of the two.

I then thought about creating a set workarea similar to the string workarea, where „working sets“ can reside during statement evaluation, but then I had the idea to simply use the string workarea for the dynamic sets as well.

I then worked on the description of the P-Code instructions at the end of this document (see there) and found out that first some work on the compiler has to be done, that is: for certain instructions (ZMX), the logic has to change. There is no need to put a target address on the stack before ZMX is executed, because the target for ZMX is in the workarea, and the next free address in the workarea is retrieved from the workarea register (which is incremented after allocation).

The instruction ZAE and ZAR are changed in a similar way; they don't have a target address any more, because the target is in the workarea. So they have only one operand (ZAE, the integer) or two (ZAR, the range), but the target for the set to be built comes from the workarea register.

ZMV doesn't change, because its target is a variable in storage, not in the working area. The instructions ZIS, ZDI and ZUN work the same as before. They don't care in which area their 2nd operand (the target) is. ZIN, too, isn't affected by this change.

The instructions ZAE, ZAR and ZMX, which put strings into the workarea, must trigger a VPO instruction at the end of the statement, so that used storage in the workarea can be freed.

I did all these changes to the compiler, and – of course – after that the (old) testcases TESTSET9 and TESTSETA don't work any more, because the PCINT interpreter needs to be adjusted to the new logic.

This has to be done in the 2nd part of the day ... let's see :-)

When this is completed (changes to the ZMX logic), the test cases TESTSET9 and TESTSETA should work again. Then ZAE and ZAR must be reworked, and then TESTSETB should work again, up to the point, where the ZUN instruction is executed (same problem as yesterday).

But: ZUN now can be changed, so that it builds a new set in the workarea, which has a range, which is the COMBINED range of the two original sets. And with that extension (hopefully), the TESTSETB testcase should finally work.

I'll tell you later ... maybe tomorrow.

28.08.2026

The statement

```
Z := [ INTVAR1 , INTVAR2 ] ;
```

now is translated to:

```
;---- Neues Statement - LCOUNTER-Page1 = 520
LOC 97
LDA 1,440                ; Z
LOD I,1,496              ; INTVAR1
ZAE I
LOD I,1,500              ; INTVAR2
ZAE I
ZUN
ZMV 52,-104
VPO 1,516
```

Compare this to the situation of two days before; no LDA statements for the target addresses of ZAE, because ZAE uses the string workarea as target space. Look at the VPO instruction at the end of the statement; this resets the string area pointer, so that allocated space (by ZAE, and maybe later by ZUN) is freed.

This works, and so do the former testcases TESTSET9 and TESTSETA, which checked set assignments and set operations like intersection, union and difference.

But I still have the same problem with ZUN; it is limited to the range of the second operand, which in this case leads to the wrong result; the second value (that of INTVAR2) is not shown. ZUN must be extended to create a new set which has the combined range of its two operands.

Let's see, what the 2nd half of the day brings (well, I have to go to a funeral service today and sing with our choir ... so maybe the compiler activity has to wait).

04.09.2026

In the meantime, the PCODE interpreter and the compiler have been enhanced and corrected, and the following results have been achieved:

- all testcases involving set operations now seem to work correctly
- this includes the examples with the new ZAE and ZAR instructions
- there have been some errors involving sets with negative components, but they have been fixed as well

Now I tried to compile the QUEENS programs, that have motivated me to do the new set implementation from the start (see the Moshix video). There are 3 variants:

- QUEENS, which is the original Moshix variant
- QUEENS1, which is my variant (only some optical changes, but same logic)
- QUEENS2, where the set with base -7 .. 7 has been changed to set of 1 .. 15

The problem is, that now we get an error 411 during compile time:

PCINT (Build 1.0 Sep 4 2026 09:13:07)

```
**** STANFORD PASCAL COMPILER, OPPOLZER VERSION OF 2024.08 ****
62             FINDQUEENS ( NEWQUEENROW + 1 , COLS + [ TRIALCOL ] ,
                               !

+++ Error P411: COMPILER IMPLEMENTATION ERROR
63             UPDIAG + [ TRIALUP ] , DOWNDIAG + [
                               !

+++ Error P411: COMPILER IMPLEMENTATION ERROR
64             TRIALDOWN ] )
                               !

+++ Error P411: COMPILER IMPLEMENTATION ERROR
**** Compiler Summary ****
****      3 Errors.
****      77 LINE(S) READ,      2 PROCEDURE(S) COMPILED,
****      271 P_INSTRUCTIONS GENERATED,      0.02 SECONDS IN COMPILATION.

**** Last Error/Warning on Line 64

**** Error/Warning Codes for this Program:
**** P411: COMPILER IMPLEMENTATION ERROR

*** EXIT Aufruf mit Parameter = 3 ***
```

This is because here sets are passed as value parameters. This is (maybe) no good programming style, CONST parameters come to mind, but anyway. For small sets it should work. Let's see how I can fix this ... more to come.

04.09.2026 (again)

The problem with the sets passed as value parameters has been fixed; in fact, an old (now obsolete) SCP instruction has been generated with the 411 error. Now this has been changed to a ZMX instruction.

The programs QUEENS, QUEENS1 and QUEENS2 now run just fine and give the correct result.

The problem detected by Moshix in the 2023 video is now finally fixed !!

But still, some problems remain:

- I wrote a program QUEENS3, where the parameters are passed by CONST, and this program (although compiling successfully), gives a wrong result (no solutions to the 8 queens problem, in fact)
- The compiler still generates an old (and obsolete) SLD instruction, in context with a standard function called CARD; this must be fixed. Should also be ZMX, IMO.
- The compiler still doesn't compile itself, which it must do, before the whole effort can be declared a success.
- And, don't forget: PCINT execution on Windows etc. is nice, but the PASCAL2 translator, which generates true 370 machine code, must be changed, too, to support the new P-Code instructions.

06.09.2026

More progress:

- The problem with the CONST parameters has been fixed
- the code sequence generated for the CONST parameters with sets is even better than before; if the set is already in the workarea (that is, if an expression is given to the const parameter), no further action is required. The address of the set in the workarea is simply passed to the called procedure
- the calling sequence for the CARD function involving the obsolete SLD instruction has been changed; it now contains a ZMX instruction, if necessary
- it turned out that the CRD P-Code instruction, which is meant to implement the CARD function, wasn't implemented in PCINT so far; this has been fixed

Next steps:

- when compiling the compiler, PCINT breaks ... this has to be fixed now
- from some Wikipedia articles, I decided to implement standard functions ALL (suggested by Wirth) and INCLUDE and EXCLUDE (present in FPC).

07.09.2026

I fixed the problem in the compiler which broke PCINT when I tried to compile the compiler; in fact, it was the de-referencing of an uninitialized pointer ... maybe some day I could enhance PCINT to throw a better exception in this case ... if the content of a pointer is far outside the allowable areas. This would be nice :-)

I then observed another minor problem; the compiler issued a syntax error 418 (compiler implementation error), when it encountered statements like

if symbol in symbolset + [semicolon] then ...

this was indeed only a minor problem of the compiler implementation :-)

After that, the compiler could compile itself, and the resulting compiler could again compile itself, and the result of this second compile was identical to the result of the first compile (which is very good). And: all set testcases still run with the expected results.

After some more polishing etc. I decided that

this compiler can be released as version 2026.09

I will inform the user community in the next days and try to build a new version on GitHub.

Stay tuned ... :-)

List of new set instructions – and their meaning

While implementing the Z (=set) instructions, I found out that there is a real problem with handling sets in the set workarea, if they have fixed positions and lengths - . especially with set unions, when the properties of the sets involved are not known at compile time (as it is the case, when the sets are built using the ZAE or ZAR instructions).

So the idea came to me that it would be best to have a set workarea without fixed positions. Instead, the sets in the workarea should be allocated „on the fly“ with the required current length.

This said, there is no need to generate LDA instructions for the places of the „working“ sets in the workarea. Instead of that, we require that the P-Code machine has a set working area which is large enough (?) and a set working area register which points to the next free usable storage in the set working area.

This, in turn, sounds very similar to the solution we already have for strings, that is: there is a string workarea of reasonable size (1 MB as default) and a pointer to the active part of the string workarea. And: there are instructions VPU and VPO which set and reset this string workarea pointer. IMO, the string workarea can (and should) be used for sets, too. Let's try ... this means that the use of the set workarea by certain instructions will trigger the generation of VPO instructions.

This section covers the new set instructions (all starting with letter Z) and their parameters and meaning.

The descriptions may have to be modified and corrected, as implementation proceeds :-)

ZAE – replaces ASE – add set element

The problem with ZAE (and ZAR) is: a new set has to be constructed in the work area, and we have no description for this set (no type), because the value of the integer, char or subrange variable which builds this set is not known at compile time.

ZAE exists in two variants, ZAE I and ZAE C. The parameter tells if the operand is of integer (or subrange of integer) type or character.

ZAE works on the topmost stack element, which is an integer (or character). It builds a set of length 5 (4 bytes prefix and length of set = 1). It constructs the offset from the given integer value (next lower value divisible by 8) and the bit offset in the byte which needs be set to one. The set is written to the string (or set) work area, the string workarea pointer is moved to SP, and then it is incremented by 8 (although the set only needs 5 bytes). A VPO instruction is triggered.

ZAR – replaces ASR – add set range

Much the same as ZAE, but it gets two operands on the stack; the lower limit of the range at SP – 4, and the upper limit at SP. The length of the resulting set can be considerably large (depends on the given range).

ZDI – replaces DIF – set difference

two sets are on top of the stack. Operand 1 is on SP, operand 2 on SP – 4. SP is reduced by 4, that is, operand 1 is popped; the result goes into operand 2. Operand2 is always large enough, the compiler ensures this. Set elements only disappear during this operation. The attributes of the result are the attributes of operand 2.

The compiler makes sure that operand2 is in the string workarea before this operation; if needed, a ZMX operation is issued before this operation.

Later it was decided that BOTH operands have to be in the set workarea before a ZDI instruction can be generated. See the description of ZUN for further details.

ZIN – replaces INN – check, if element is in set

The set address is on the topmost stack position, the value of the element is on the SP – 4. Length and offset of the set are retrieved from the meta data of the set (first four bytes). The value of the element is examined and checked for existence; then one stack element (the set address) is popped and the then topmost stack element is replaced by the boolean value „found“ or „not found“.

ZIS – replaces INT – set intersection

see ZDI. Works much the same, but the bit handling of the sets is different.

Operand 2 must be in the string workarea, but because ZIS is symmetric with respect to its operands, it would be OK to have a variant with exchanged operands (that is, if operand 1 is in the workarea and not operand2, a special variant of ZIS could cope with this situation without requiring an additional ZMX instruction).

Later it was decided that BOTH operands have to be in the set workarea before a ZIS instruction can be generated. See the description of ZUN for further details.

ZMV – replaces SMV – move set

the source set has its length and offset in its meta data (first four bytes); the values for the target set are part of the ZMV instruction. Move set from source to target.

lent = length of target (from instruction)

offst = offset of target (from instruction)

lens = length of source (from source meta data)

offss = offset of source (from source meta data)

The addresses are fetched from the stack; sequence depends on the sign of the length; if length < 0, then the target is at SP and the source at SP – 4, otherwise target at SP – 4 and source at SP. The length is made positive.

Now we have the lengths and offsets of the operands. The delta of the offsets is computed. The resulting transfer len and the resulting transfer offsets for source and target are computed. It is possible that the resulting transfer len is zero or negative; in this case there is no transfer and the result will be the empty set.

Later we will have to implement checks for elements which cannot be stored into the target set; a runtime error (range violation) should be reported in such cases.

ZMX – ZMV plus

see places where ZMX is generated; maybe there is no need to provide a target address beforehand – change implementation and description – use string workarea, trigger VPO.

ZMX is the same as ZMV, but the address of the target stays on the stack for further processing.

ZUN – replaces UNI – set union

see ZDI (set difference) and ZIS (set intersection). The problem here is, that elements do not only disappear from the result set; the result set can be larger than the two operand sets, so the implementation is somehow harder than ZDI's and ZIS's. The description (length and offset) of the result set must be computed from the two operand descriptions in a more complicated way.

See if the result of ZUN fits into operand 2, which has to be in the workarea like in the other instructions ... if yes, there is no need to create a new set in the workarea, the result can be written into operand2. If not, a new set result must be build in the workarea, and VPO must be triggered. This all can only be decided at runtime, which means, that VPO must be triggered anyway.

ZUN checks the lengths and offsets of the two set operands and then decides, if the result fits into operand 2. If this is true, operand 2 is the target of the operation (which is already in the workarea), otherwise a new set is built in the workarea, which is large enough for the result. The result is preformatted with the larger of the two operands and then the other operand is bit-operated into it.

Like ZIS, the ZUN operation is symmetric and would run better, if the first operand could qualify for a possible target of the operation, too ... for example, if the first operand is already in the workarea and it is the larger set OR if the second operand is not in the workarea yet; in both cases a variant with exchanged operands could help.

Late in the development process it was decided that both operands of ZUN have to be in the set workarea, before the ZUN instruction can be generated. The reason is that in certain circumstances the operand 1 field can be the target of the ZUN operation, and it is very bad if operand 1 is a set variable which is not meant to change during the ZUN operation. The problem occurred especially in the context of CONST parameters (with ZUN operations in them).

The implementation of ZUN in PCINT maybe should be further optimized, so that it really is completely symmetric with respect to the two operands.

CRD – cardinality of sets

This instruction does not begin with Z, but is belongs to the set-related instructions, too. It implements the standard function CARD, and ... as it turned out ... it was not implemented in PCINT so far.

CRD fetches the set (its argument) from the topmost stack position, computes its cardinality (the actual number of elements in the set) and replaces the topmost stack element with this number. The stack pointer does not change.

The problem with this instruction was: it generated an (now obsolete) SLD instruction, which had to be replaced (that is: omitted).

I hope you like the new set implementation of New Stanford Pascal;
please send comments and suggestions to

berndoppolzer@yahoo.com

or

bernd.oppolzer@t-online.de